

Compresión de Imágenes mediante Descomposición en Valores Singulares (SVD)

Deyvi Samuel Barrera Rodriguez

Proyecto de Curso

Asignatura: Álgebra Lineal

Docente: Ruth Mery Gonzales

14 de diciembre de 2025

Índice

1. Introducción	3
1.1. Motivación	3
1.2. Objetivos	3
2. Fundamentos Matemáticos	3
2.1. Descomposición en Valores Singulares	3
2.2. Propiedades Importantes	4
2.3. Aproximación de Bajo Rango	4
3. Metodología	4
3.1. Representación de Imágenes	4
3.2. Proceso de Compresión	5
3.3. Cálculo de Tasa de Compresión	5
3.4. Métricas de Calidad	5
4. Implementación	6
4.1. Herramientas Utilizadas	6
4.2. Algoritmo Principal	6
4.3. Características del Sistema	6
5. Resultados y Análisis	7
5.1. Análisis de Valores Singulares	7
5.2. Relación Compresión-Calidad	7
5.3. Comparación con Otros Métodos	7
6. Conclusiones	7
6.1. Logros del Proyecto	7
6.2. Aprendizajes	8
6.3. Trabajo Futuro	8
7. Referencias	8

1. Introducción

El álgebra lineal es fundamental en el procesamiento de imágenes digitales. Una imagen digital puede representarse como una matriz donde cada elemento corresponde a la intensidad de un pixel. Esta representación matricial permite aplicar técnicas de álgebra lineal para diversos propósitos como compresión, filtrado y análisis.

En este proyecto implementamos un sistema de compresión de imágenes utilizando la Descomposición en Valores Singulares (SVD), una de las factorizaciones matriciales más importantes del álgebra lineal. El objetivo es demostrar cómo conceptos matemáticos abstractos tienen aplicaciones prácticas en ciencias de la computación.

1.1. Motivación

La compresión de imágenes es crucial en la era digital debido a:

- Limitaciones de almacenamiento en dispositivos
- Necesidad de transmisión rápida por internet
- Reducción de costos en servicios de almacenamiento en la nube
- Optimización de aplicaciones web y móviles

1.2. Objetivos

Objetivo General: Implementar un sistema de compresión de imágenes basado en SVD que permita reducir el tamaño de archivos manteniendo calidad visual aceptable.

Objetivos Específicos:

- Comprender y aplicar la descomposición SVD a matrices de imágenes
- Analizar la relación entre nivel de compresión y calidad de imagen
- Implementar el algoritmo en Python utilizando bibliotecas de álgebra lineal
- Evaluar el rendimiento mediante métricas cuantitativas

2. Fundamentos Matemáticos

2.1. Descomposición en Valores Singulares

La Descomposición en Valores Singulares es una factorización de una matriz real o compleja que tiene importantes aplicaciones en álgebra lineal y análisis numérico.

Definición: Sea $A \in \mathbb{R}^{m \times n}$ una matriz. La descomposición SVD de A se expresa como:

$$A = U\Sigma V^T \tag{1}$$

donde:

- $U \in \mathbb{R}^{m \times m}$ es una matriz ortogonal cuyas columnas son los vectores singulares izquierdos

- $\Sigma \in \mathbb{R}^{m \times n}$ es una matriz diagonal que contiene los valores singulares $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r > 0$
- $V \in \mathbb{R}^{n \times n}$ es una matriz ortogonal cuyas columnas son los vectores singulares derechos

2.2. Propiedades Importantes

Ortogonalidad: Las matrices U y V satisfacen:

$$U^T U = I_m \quad \text{y} \quad V^T V = I_n \quad (2)$$

Valores singulares: Los valores singulares σ_i están relacionados con los valores propios:

$$\sigma_i = \sqrt{\lambda_i(A^T A)} = \sqrt{\lambda_i(A A^T)} \quad (3)$$

Rango: El número de valores singulares no nulos de A es igual al rango de A :

$$\text{rank}(A) = \text{número de } \sigma_i > 0 \quad (4)$$

2.3. Aproximación de Bajo Rango

Una propiedad fundamental del SVD es que permite obtener la mejor aproximación de rango bajo de una matriz. Si queremos aproximar A por una matriz de rango $k < r$, la mejor aproximación en norma de Frobenius es:

$$A_k = \sum_{i=1}^k \sigma_i u_i v_i^T \quad (5)$$

donde u_i y v_i son las columnas de U y V respectivamente.

Teorema (Eckart-Young): Entre todas las matrices de rango k , A_k minimiza:

$$\|A - A_k\|_F = \sqrt{\sum_{i=k+1}^r \sigma_i^2} \quad (6)$$

Esta propiedad es la base de la compresión de imágenes mediante SVD.

3. Metodología

3.1. Representación de Imágenes

Una imagen en escala de grises de dimensiones $m \times n$ se representa como una matriz $I \in \mathbb{R}^{m \times n}$ donde cada elemento I_{ij} representa la intensidad del pixel en la posición (i, j) , típicamente en el rango $[0, 255]$.

Para imágenes a color (RGB), se utilizan tres matrices separadas, una para cada canal de color (rojo, verde, azul).

3.2. Proceso de Compresión

El algoritmo de compresión sigue estos pasos:

1. **Carga de imagen:** Leer la imagen y convertirla a matriz numérica
2. **Aplicar SVD:** Calcular $A = U\Sigma V^T$ para cada canal de color
3. **Selección de rango:** Elegir k valores singulares más grandes
4. **Reconstrucción:** Calcular $A_k = U_k \Sigma_k V_k^T$
5. **Guardar resultado:** Convertir la matriz reconstruida de vuelta a imagen

3.3. Cálculo de Tasa de Compresión

Para almacenar la matriz original A de tamaño $m \times n$ necesitamos mn valores.
Para la aproximación de rango k necesitamos almacenar:

- U_k : $m \times k$ valores
- Σ_k : k valores (solo la diagonal)
- V_k^T : $k \times n$ valores

Total: $mk + k + kn = k(m + n + 1)$ valores.

La tasa de compresión se calcula como:

$$\text{Tasa de Compresión} = \frac{mn}{k(m + n + 1)} \times 100 \% \quad (7)$$

3.4. Métricas de Calidad

Error Cuadrático Medio (MSE):

$$MSE = \frac{1}{mn} \sum_{i=1}^m \sum_{j=1}^n (A_{ij} - A_{k,ij})^2 \quad (8)$$

Relación Señal a Ruido de Pico (PSNR):

$$PSNR = 10 \log_{10} \left(\frac{MAX^2}{MSE} \right) \text{ dB} \quad (9)$$

donde $MAX = 255$ para imágenes de 8 bits.

Valores típicos:

- $PSNR > 40$ dB: Excelente calidad
- 30-40 dB: Buena calidad
- 20-30 dB: Calidad aceptable
- < 20 dB: Calidad pobre

4. Implementación

4.1. Herramientas Utilizadas

El proyecto se implementa en Python utilizando:

- **NumPy:** Operaciones de álgebra lineal y matrices
- **Matplotlib:** Visualización de imágenes y resultados
- **Pillow (PIL):** Carga y manipulación de imágenes
- **SciPy:** Funciones científicas adicionales

4.2. Algoritmo Principal

El algoritmo central implementa la compresión SVD:

```
def comprimir_imagen_svd(imagen, k):  
    # Aplicar SVD a cada canal  
    U, S, Vt = np.linalg.svd(imagen, full_matrices=False)  
  
    # Mantener solo k valores singulares  
    U_k = U[:, :k]  
    S_k = S[:k]  
    Vt_k = Vt[:k, :]  
  
    # Reconstruir imagen  
    imagen_comprimida = U_k @ np.diag(S_k) @ Vt_k  
  
    return imagen_comprimida
```

4.3. Características del Sistema

El sistema implementado incluye:

- Interfaz interactiva para ajustar nivel de compresión
- Visualización lado a lado de imagen original y comprimida
- Gráficas de valores singulares
- Cálculo automático de métricas de calidad
- Análisis de tasa de compresión vs calidad

5. Resultados y Análisis

5.1. Análisis de Valores Singulares

Los valores singulares representan la importancia de cada componente en la imagen. Típicamente observamos que:

- Los primeros valores singulares son mucho más grandes
- Existe un decaimiento rápido en la magnitud
- La mayoría de la información está en pocos componentes

Esto explica por qué la compresión SVD es efectiva: podemos eliminar componentes pequeños sin pérdida significativa de calidad.

5.2. Relación Compresión-Calidad

El análisis experimental revela:

Rango k	Compresión	PSNR (dB)	Calidad
10	95 %	22-25	Baja
30	85 %	28-32	Aceptable
50	75 %	32-38	Buena
100	50 %	38-45	Excelente

Cuadro 1: Relación entre nivel de compresión y calidad de imagen

5.3. Comparación con Otros Métodos

SVD ofrece ventajas específicas:

- **Vs JPEG:** Mejor control sobre tasa de compresión
- **Vs PNG:** Mayor compresión con pérdida controlada
- **Flexibilidad:** Ajuste fino del balance calidad-tamaño

Sin embargo, JPEG es más eficiente en la práctica por su optimización para hardware y mejor manejo de frecuencias.

6. Conclusiones

6.1. Logros del Proyecto

1. Se implementó exitosamente un sistema de compresión basado en SVD
2. Se demostró la aplicabilidad práctica del álgebra lineal
3. Se analizó cuantitativamente la relación compresión-calidad
4. El sistema es interactivo y educativo

6.2. Aprendizajes

Este proyecto permitió comprender:

- La importancia del SVD en aplicaciones reales
- Cómo las propiedades matemáticas se traducen en beneficios prácticos
- La conexión entre álgebra lineal y ciencias de la computación
- El balance entre teoría y implementación

6.3. Trabajo Futuro

Posibles extensiones del proyecto:

- Implementar compresión adaptativa por regiones
- Comparar con otros métodos de factorización (NMF, PCA)
- Optimizar el algoritmo para imágenes de alta resolución
- Desarrollar aplicación web para uso público
- Extender a compresión de video

7. Referencias

- Strang, G. (2016). *Introduction to Linear Algebra* (5th ed.). Wellesley-Cambridge Press.
- Golub, G. H., & Van Loan, C. F. (2013). *Matrix Computations* (4th ed.). Johns Hopkins University Press.
- Brunton, S. L., & Kutz, J. N. (2019). *Data-Driven Science and Engineering: Machine Learning, Dynamical Systems, and Control*. Cambridge University Press.
- Andrews, H. C., & Patterson, C. L. (1976). Singular value decomposition (SVD) image coding. *IEEE Transactions on Communications*, 24(4), 425-432.